ZARGUX: ORIGIN-BASED INCIDENT DETECTION FOR LINUX SERVERS

PROJECT SYNOPSIS

A report submitted in partial fulfillment of the requirements

for the degree of Bachelor of Technology in

Computer Science & Engineering / Cyber Security

Submitted by

Abhyuday Pundir — 2400320100059

Under the guidance of

Sarath G, Cybersecurity Trainer & Consultant

Department of Computer Science & Engineering

ABES Engineering College, Ghaziabad

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Session 2026–27

1. Introduction / Background

Attackers increasingly drop malicious binaries (cryptominers, backdoors) into writable, non-persistent locations such as /dev/shm and /tmp and execute them directly, bypassing the software installation process entirely. Traditional Linux monitoring tools flag high CPU or memory usage without asking where the responsible binary actually came from, so a malicious process generating the same load as a legitimate application is indistinguishable from it.

2. Problem Statement

Existing load-threshold monitoring cannot tell a legitimate CPU-heavy application apart from a malicious process causing identical load, because both are judged purely on resource consumption rather than on where their executable actually lives and whether it can be traced back to an installed, accountable package.

3. Objectives

Simulate a drop-and-run attack by copying a binary into a writable temporary location (/dev/shm or /tmp) and executing it there, alongside a legitimate baseline run of the same binary from its installed location.

Detect the attack by resolving each running process's binary path via /proc/<pid>/exe and classifying it against dpkg package ownership and known writable-temp path prefixes, independent of CPU load.

Correlate detection with real-time CPU pressure (Linux PSI, /proc/pressure/cpu) to indicate severity.

Alert and persist every finding (timestamp, evidence, severity) to a growing, queryable record.

Demonstrate a working detector wired to a live, auto-refreshing web dashboard, with a negative control proving the detector does not blindly flag every unpackaged binary.

4. Scope of the Project

In scope: origin-based classification of running processes (writable-temp-path detection, deleted-binary detection, dpkg ownership lookup), CPU pressure correlation via PSI, persistent finding storage, and a live browser dashboard. Out of scope: network-based attack detection, authentication/brute-force monitoring, automated response actions such as killing or isolating a process (logged as future work), and production-grade multi-host deployment.

5. Proposed System / Methodology

The system follows an Attack → Evidence → Detection → Response pipeline.

Attack: a binary (stress-ng) is copied into /dev/shm or /tmp and launched via systemd-run, simulating how real malware drops and executes from a writable location; a legitimate run from /usr/bin and a legitimate-but-unpackaged negative control from ~/bin are run alongside it.

Evidence: /proc/<pid>/exe (resolved binary path) and /proc/pressure/cpu (system-wide CPU pressure) are read for each process.

Detection: the classify() routine in zargux.py flags a process HIGH if its path starts with /dev/shm/, /tmp/, /var/tmp/, or has been deleted from disk; otherwise it checks dpkg ownership and returns WATCH (unpackaged, but not a writable-temp path) or CLEAR (package-managed).

Response: each finding is timestamped and persisted via save_finding() to findings.json, which a live HTML dashboard polls every 3 seconds and color-codes by severity.

[Architecture diagram to be inserted here — boxes: Attacker/Simulator → Target VM → Detector (zargux.py) → findings.json → Dashboard.]

6. Modules / Module Description

Attack Simulator (attack.sh, systemd-run based process launcher) — built by Abhyuday Pundir.

Detector / Origin Classifier (zargux.py: scan, watch, and demo subcommands) — built by Abhyuday Pundir.

Evidence & Persistence Layer (save_finding() writing to findings.json) — built by Abhyuday Pundir.

Live Dashboard (index.html, served via Python's built-in HTTP server) — built by Abhyuday Pundir.

Solo project — all modules built by one student.

7. Hardware & Software Requirements

Host: Fedora Linux (development machine). Target: Ubuntu 24.04 LTS virtual machine via Multipass 1.16.4 (2 vCPU, 2 GB RAM, 15 GB disk).

Languages: Python 3 (detector/CLI prototype), Zig 0.13.0 (planned production collector, cross-compiled to a static musl binary).

Tools: stress-ng (attack simulation), systemd-run (process lifecycle management), dpkg (package ownership lookup), Linux PSI (/proc/pressure/cpu).

Frontend: plain HTML/CSS/JavaScript, served with Python's http.server — no additional frameworks.

8. Feasibility Study

Technical: every component relies on tooling already present on a standard Linux distribution (Python standard library, systemd, dpkg, the kernel's PSI interface), so no new frameworks or external services are required; feasibility is proven by the working prototype itself.

Economic: zero additional cost — only open-source, freely available tools are used, and the VM runs comfortably within a 2 GB RAM budget.

Operational: the detector is a single Python script plus one static HTML file, so it can be copied onto any modern Linux server and run immediately, without installation steps beyond what the server already has.

9. Expected Outcomes

A detector that correctly classifies a live process as CLEAR, WATCH, or HIGH based on binary origin rather than CPU load.

Verified correct behaviour on three test cases: a legitimate high-CPU run (CLEAR, no alert), an identical binary dropped into /dev/shm and run (HIGH, alerted), and a legitimate but unpackaged script in a normal location (WATCH, not falsely flagged HIGH).

A live dashboard that shows a newly triggered finding within approximately 3 seconds.

Zero false positives observed on the negative control across repeated runs, with saved evidence files backing every stage.

10. Timeline / Project Plan

Day 1: VM setup (Multipass, Ubuntu 24.04), toolchain setup (Zig, Python), confirmed PSI availability, finalised project definition and attack model.

Day 2: built and verified the attack simulation (legitimate run vs. /dev/shm copy), captured readlink and PSI evidence for both.

Day 3: built and validated the detection engine (zargux.py) against three test cases (legitimate, dropped, negative control); wired the persistence layer and live dashboard end to end.

Day 4: prepared documentation (Synopsis, SRS, Use-Cases, PPT) and rehearsed the live demo.

11. References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[2] The Linux Kernel Documentation, “Pressure Stall Information (PSI),” https://docs.kernel.org/accounting/psi.html

[3] dpkg(1) manual page, Debian Project, https://man7.org/linux/man-pages/man1/dpkg.1.html

[4] systemd-run(1) manual page, freedesktop.org, https://www.freedesktop.org/software/systemd/man/systemd-run.html

[5] Zig Software Foundation, “Zig 0.13.0 Documentation,” https://ziglang.org/documentation/0.13.0/